Applied Soft Computing Journal 94 (2020) 106433

Contents lists available at ScienceDirect

Applied Soft Computing Journal

journal homepage: www.elsevier.com/locate/asoc

A novel randomized machine learning approach: Reservoir computing extreme learning machine

Ömer Faruk Ertuğrul

Department of Electrical and Electronic Engineering, Batman University, 72060 Batman, Turkey

ARTICLE INFO

Article history:
Received 19 July 2019
Received in revised form 9 January 2020
Accepted 24 May 2020
Available online 30 May 2020

Keywords:
Reservoir computing
Extreme learning machine
Randomization in machine learning
Randomized artificial neural network

ABSTRACT

In this study, a novel approach that is based on reservoir computing, which is a successful method in modeling sequential datasets, and extreme learning machines, which has a high generalization capacity, was proposed to model a non-sequential dataset or system. The proposed approach does not require any optimization stage; each weight (except weights in the output layer), biases, the number of neurons in the reservoir, activation functions and the parameters of activation functions were determined arbitrarily and the weights in the output layer were calculated based on these arbitrarily assigned parameters. The proposed approach was evaluated and validated with 60 different benchmark datasets. Obtained results were compared with literature findings and results obtained by each of the extreme learning machine (ELM), randomized artificial neural network, random vector functional link, stochastic ELM, and pruned stochastic ELM methods. Achieved results are successful enough to be employed in classification and regression.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

It is clear that humanity has high ability in learning and adapting himself [1]. Therefore, many machine learning methods have been proposed in order to model human learning system, such as an artificial neural network (ANN) [2], generalized behavioral learning [3], fuzzy logic [4] and metaheuristic methods [5–8]. Although the structure of a neural network is too complex, ANN attempts to model it artificially. To achieve successful learning many different types of learning approaches have been proposed and employed. In general, the structure of proposed ANNs goes in a different way of the original idea, modeling the human neural network. For instance, (1) in human neural network, there are many chaotic links between neurons, (2) there is not any layer structure, and (3) there is not any change that each of the neurons to have the same transfer/activation function [9–13].

In reservoir computing (RC), which is a recurrent type artificial neural network [14], there are randomized links between neurons in the reservoir and there is not any layer structure, but it employs the same activation function for each neuron. Although, there is a large and growing literature about the success of RC [15–20] and many variants of RC have been proposed [21–27], as seen in the literature, RC shows really great success in only time based datasets, since it is a recurrent type ANN (i.e., it has short-term memory [28]), it cannot be employed in other types of datasets.

On the other hand, extreme learning machine, which is a training method for one single hidden layer ANN (SLFN), becomes popular day by day because of its generalization capacity, mathematical simplicity and extremely fast training stage [29]. Regardless of the fact that, in ELM, the weights and biases are assigned randomly, while the weights in the output layer calculated analytically, it still does not show the same randomization capacity of RC. It requires optimizing the number of neurons in the hidden and activation functions to achieve acceptable success rates [30]. In my previous work, two novel variants of ELM, which are stochastic ELM (S-ELM) and pruned stochastic ELM (PS-ELM), were proposed [31]. It was shown that high success can also be achieved with using a randomly assigned number of neurons in the hidden layer and also an arbitrarily assigned activation function for each of the neurons in the hidden layer by employing parallel networks based on a winner takes all strategy [31].

In this paper, the investigation of achieving a more realistic method is continuing. Instead of employing parallel networks, an acceptable success rate can be achieved by assigning each of the parameters of an ANN randomly and only learning by the weight in the output layer. In order to have a more realistic ANN, the features of RC, S-ELM and ELM were combined together and a novel approach, which can be called as reservoir comping — ELM (RC-ELM), has proposed. In the literature, there are some different attempts to combine these methods [14,37–40], but each of them was built on learning/modeling time-ordered (sequential) datasets. In this paper, these methods have been combined together in a way that it can be employed in non-sequential datasets. To the author's best knowledge, there is not

E-mail address: omerfaruk.ertugrul@batman.edu.tr.

https://doi.org/10.1016/j.asoc.2020.106433
1568-4946/© 2020 Elsevier B.V. All rights reserved.

Table 1
Properties of employed datasets.

Name	Type	Performed task	#Attributes	#Observations	#Classes	Source
Lithuanian	Synthetic	Classification	2	1000	2	[32]
Highleyman	Synthetic	Classification	2	1000	2	[32]
Banana shaped	Synthetic	Classification	2	1000	2	[32]
Spherical	Synthetic	Classification	2	1000	2	[32]
Multi-class	Synthetic	Classification	2	1000	8	[32]
Liver	Real	Classification	7	345	2	[33]
Pima Indian diabetes	Real	Classification	8	762	2	[33]
Hepatitis	Real	Classification	19	150	2	[33]
Banana	Real	Classification	2	5300	2	[34]
Image segmentation	Real	Classification	19	2315	7	[33]
Satellite image	Real	Classification	36	6435	7	[33]
Statlog (shuttle)	Real	Classification	9	58000	7	[33]
Abalone	Real	Classification	8	4177	29	[33]
Wine	Real	Classification	13	178	3	[33]
Breast tissue	Real	Classification	10	106	4	[33]
Cardiotocography	Real	Classification	22	2126	3	[33]
Skin segmentation	Real	Classification	3	245057	2	[33]
Seeds	Real	Classification	7	210	3	[33]
EEG eye state	Real	Classification	15	14980	2	[33]
Seismic bumps	Real	Classification	18	2584	2	[33]
Banknote authentication	Real	Classification	4	1372	2	[33]
Balance scale	Real	Classification	4	625	3	[33]
Acute inflammations	Real	Classification	7	120	2	[33]
Dermatology	Real	Classification	35	366	6	[33]
Diabetic retinopathy debrecen	Real	Classification	19	1151	2	[33]
Fertility	Real	Classification	9	100	2	[33]
Glass	Real	Classification	10	214	2	[33]
Haberman	Real	Classification	3	306	2	[33]
Hayes-Roth	Real	Classification	5	132	3	[33]
QSAR biodegradation	Real	Classification	41	1055	2	[33]
Approximate sinc	Synthetic	Regression	1	5000	–	[35]
Forest fire	Real	Regression	13	517	–	[33]
CASP 5–9	Real	Regression	9	45730	–	[33]
Istanbul stock exchange	Real	Regression	7	537	–	[33]
Ailerons	Real	Regression	40	13750	–	[36]
Delta ailerons	Real	Regression	5	7129	–	[36]
Auto-price	Real	Regression	15	159	–	[33]
Bank-8FM	Real	Regression	8	6481	–	[36]
Boston housing	Real	Regression	14	506	–	[33]
Breast cancer	Real	Regression	32	194	–	[33]
California housing	Real	Regression	9	26940	–	[36]
Census-8L	Real	Regression	8	22784	–	[36]
Census-8H	Real	Regression	8	22784	–	[36]
Census-16L	Real	Regression	16	22784	–	[36]
Census-16H	Real	Regression	16	22784	–	[36]
CPU-small	Real	Regression	12	8192	–	[33]
CPU	Real	Regression	21	8192	–	[33]
Diabetes child	Real	Regression	2	43	–	[36]
Delta elevators	Real	Regression	6	9517	–	[36]
Elevators	Real	Regression	18	16599	–	[36]
Kinematics	Real	Regression	8	8192	–	[36]
Machine-CPU	Real	Regression	6	209	–	[36]
Puma-8NH	Real	Regression	8	6677	–	[36]
Puma-32H	Real	Regression	32	4938	–	[36]
Pyrimidines	Real	Regression	28	74	–	[36]
Servo	Real	Regression	4	167	–	[33]
Stocks	Real	Regression	9	950	–	[36]
Triazines	Real	Regression	60	186	–	[36]
Energy efficiency: cooling	Real	Regression	8	768	–	[33]
Energy efficiency: heating	Real	Regression	8	768	–	[33]

any other attempt or variant of RC in the literature that employed in non-sequential datasets.

Furthermore, it can be seen from the literature that optimizing network is a major task in order to achieve success and generally, the optimal network structure is determined by trials [41–45]. In the proposed approach, each of the parameters (except weights in the output layer) and the also structure of the network were assigned arbitrarily. The contributions of this paper can be listed as follows.

1. RC and ELM were combined together to achieve a faster training speed,
2. Each of the variants of the RC has been proposed to be employed in sequential datasets, but this proposed RC-ELM can be employed in non-sequential datasets,
3. Determining optimum parameters or structure of a machine learning method is a very time-consuming process. On the other hand, in the proposed approach, each parameter of the proposed approach can be assigned arbitrary,
4. Different activation functions with different parameters can be assigned for each of the neurons in the reservoir,

In order to validate these contributions and test the success of the proposed approach, 60 benchmark datasets were employed according to the given methodology in Section 2. Obtained results

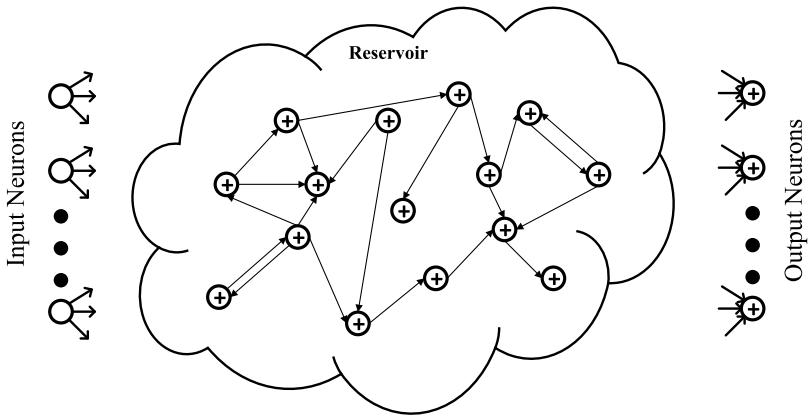

Fig. 1. Reservoir Computing.

were given and discussed in Sections 3 and 4, respectively, and the last section was concluded the paper.

2. Materials and method

2.1. Material

In order to evaluate and validate the proposed approach 30 benchmark classification datasets and 30 benchmark regression datasets have been employed, and details about these datasets were given in Table 1 and also their details can be found in [31, 33,46].

2.2. Reservoir computing

Reservoir computing is a recurrent type artificial neural network. In RC, there is a reservoir that has a number of neurons inside. A typical RC network can be visualized in Fig. 1.

In RC, a group of neurons was created and linked randomly in the hidden layer as seen in Fig. 1. These an arbitrary created and linked group of neurons in the hidden layer is called reservoir [47]. This reservoir remains constant during training and test stages. The outputs are calculated based on this randomly generated reservoir. Having the ability in estimating temporal variables is one of the major property of RC and it can be said that this method was built on learning/modeling time-based/sequential systems/datasets. The reservoir state, output, and weights in the output layer in the RC can be calculated as follows [28].

$$x(n+1) = f(W^{in}u(n+1) + Wx(n) + W^{fb}y(n)) \quad (1)$$

$$y(n) = g(W^{out}[x(n) : u(n)]) \quad (2)$$

$$W^{out} = YX^T\left(XX^T + \zeta I\right)^{-1} \quad (3)$$

where u, y, x, n, W^{in}, W, W^{fb}, W^{out}, f, g, ζ, I, ^{T}, and $^{-1}$ are input, output, reservoir state, instant, weight in the input, reservoir, feedback, and output layer, sigmoid function in the reservoir, output activation function, regularization coefficient, identity matrix, transpose of a matrix and inverse of a matrix, respectively. The weights in the output layer can be calculated by ridge regression or the pseudoinverse method [28].

2.3. Extreme Learning Machines (ELM)

ELM is a training method, whose training speed is extremely fast, in a single hidden feed-forward artificial neural network (SLFN) as seen in Fig. 2 [29].

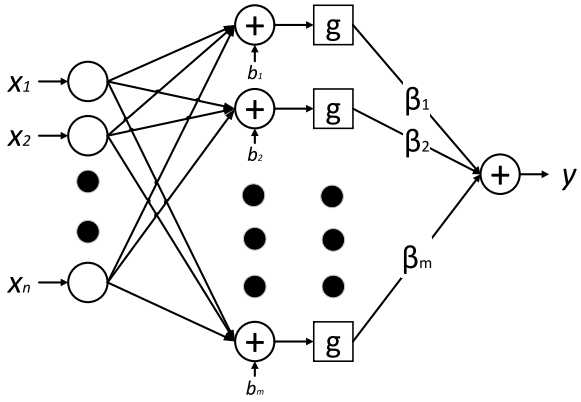

Fig. 2. The structure of the single hidden layer feed-forward artificial neural network.

The output of the SLFN can be calculated as follows [29].

$$y = \sum_{j=1}^{m} \beta_j g\left(\sum_{i=1}^{n} w_{i,j}x_i + b_j\right) \quad (4)$$

where x, y, n, m, $g(.)$, $w_{i,j}$, b_j, and β_j are the input, output, the number of features in the input, the number of neurons in the hidden layer, activation function, weights in the input layer, biases, and the weights in the output layer, respectively. In ELM, the weights in the input layer and biases are assigned arbitrarily, while weights in the output layer are calculated analytically. The output of the jth neuron in the hidden layer is:

$$H_j = w_j.x + b_j \quad (5)$$

where $w_j = [w_{1,j}\ w_{2,j}\ \cdots\ w_{n,j}]^T$ and $x = [x_1,\ x_2\ \cdots\ x_n]^T$. Eq. (5) can be rewritten as follows.

$$g(H)\beta = y \quad (6)$$

Since $g(H)$ is not in a square matrix, its inverse can be obtained by Moore–Penrose generalized inverse method in order to determine the weights in the output layer ($\beta_{1\cdots m}$) as follows.

$$\widehat{\beta} = g(H)^{+}y \quad (7)$$

where $g(H)^{+}$ is the generalized Moore–Penrose generalized inverse of $g(H)$ matrix [29,48–51].

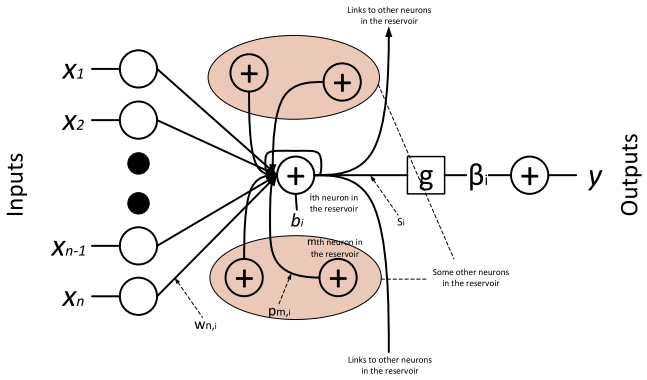

Fig. 3. Summation of the ith neuron in the reservoir.

2.4. Reservoir Computing based Extreme Learning Machines (RC-ELM)

Traditional reservoir computing methods are based on delays [21–27]. Therefore, they showed high success in the time-based datasets. In this paper, a novel reservoir computing approach was proposed. The structure of the proposed approach is the same as given in Fig. 1. In the proposed approach a neuron in the hidden, which has both input and output links can be seen in Fig. 3.

The summation of the ith neuron in the reservoir can be calculated as follows.

$$S_i = \boldsymbol{w}_{\boldsymbol{1\cdots n,i}}\boldsymbol{X} + \boldsymbol{p}_{\boldsymbol{1\cdots m,i}}\boldsymbol{S}_{\boldsymbol{1\cdots m}} + b_i \tag{8}$$

where S is the summation of inputs of a neuron in the reservoir, while X is the input. w, p, b, n, and m are weights in the input layer, weights of the neurons in the reservoir, bias, number of inputs, and number of neurons in the reservoir, respectively. The summation of inputs of a neuron in the reservoir can be calculated as follows.

$$S_i - \boldsymbol{p}_{\boldsymbol{1\cdots m,i}}\boldsymbol{S}_{\boldsymbol{1\cdots m}} = \boldsymbol{w}_{\boldsymbol{1\cdots n,i}}\boldsymbol{X} + b_i \tag{9}$$

Each of the summations of inputs of a neuron in the reservoir can be calculated by linear algebra as follows.

$$\boldsymbol{S}_{\boldsymbol{1\cdots m}}(1 - \boldsymbol{p}_{\boldsymbol{1\cdots m,1\cdots m}}) = \boldsymbol{w}_{\boldsymbol{1\cdots n,1\cdots m}}\boldsymbol{X} + \boldsymbol{b}_{\boldsymbol{1\cdots m}} \tag{10}$$

$$\boldsymbol{S}_{\boldsymbol{1\cdots m}} = inv\left(1 - \boldsymbol{p}_{\boldsymbol{1\cdots m,1\cdots m}}\right) * \boldsymbol{w}_{\boldsymbol{1\cdots n,1\cdots m}}\boldsymbol{X} + \boldsymbol{b}_{\boldsymbol{1\cdots m}}$$

As seen from Fig. 3, the links between neurons in the reservoir are generated before activation function, otherwise, the calculating the summations of inputs of a neuron in the reservoir will be a really complex issue. Later the output weights were calculated by the Moore–Penrose generalized inverse method.

$$\widehat{\beta} = g(S)^{+}y \tag{11}$$

where $g(S)^{+}$ is the generalized Moore–Penrose generalized inverse of $g(S)$ matrix. Here, some other additional properties, which are given below, were added to the traditional randomized feed-forward artificial neural network and the reservoir computing methods.

1. The number of neurons in the reservoir can be determined both randomly or by the user/researcher.

2. The weights in the input layer were determined randomly similar to other randomized ANNs, but in RC-ELM, these weights may be assigned as "0" based on the ratio of the linked and unlinked neurons in the input layer and reservoir. I.e., having "0" as weight means there is not a link between that neuron in the input layer and that neuron in the reservoir. The ratio of the linked and unlinked neurons in the input layer and reservoir can be determined both arbitrarily or by the user/researcher. This is based on the literature finding that the weights in the input layer can be assigned randomly without reducing the success rate [29,41,48,49,52–56].

3. The weights of the neurons in the reservoir were determined randomly. Therefore, there may not be any link between each neuron in the reservoir. The ratio of the linked and unlinked neurons in the reservoir can be determined both arbitrarily or by the user/researcher.

4. The biases of the neurons in the reservoir were determined randomly, similar to other randomized ANNs.

5. Since Hornik proved that arbitrarily bounded and non-constant activation functions can be employed as an activation function [57]. An activation function, which may have arbitrarily assigned or a user/researcher defined parameters (see a and b in Equation 12–16), can be assigned both randomly or by a user/researcher for only the links between neurons in the reservoir and the neurons in the output layer. Some activation functions, which are sigmoid, sine, Gaussian, hard-limit, and multi-quadratic functions are given below.

$$g\,(a,b,x) = \frac{1}{1+e^{-(ax+b)}} \tag{12}$$

$$g\,(a,b,x) = \sin(ax+b) \tag{13}$$

$$g\,(a,b,x) = e^{-a\|x-b\|} \tag{14}$$

$$g\,(a,b,x) = \begin{cases} 1, & if\ ax+b \le 0 \\ 0, & otherwise \end{cases} \tag{15}$$

$$g\,(a,b,x) = \sqrt{\|x-a\|^2 + b^2} \tag{16}$$

Since any piecewise of a continuous function can be used as a transfer/activation function [29], by assigning a and b values in an employed activation function, higher nonlinearities between input(s) and output(s) can be achieved [31].

6. As expressed previously, in order to achieve acceptable success rates in ANNs, the optimal number of neurons in the network and employed activation function must be optimized [58–60]. On the other hand, in RC-ELM, a number of neurons in the reservoir, weights in the input layer and reservoir, the ratios of the linked and unlinked neurons in each part of the network can be determined both arbitrarily or by the user/researcher.

The optimal values of the number of neurons in the reservoir, the ratio of the linked and unlinked neurons in the input layer and reservoir, the ratio of the linked and unlinked neurons in the reservoir, the activation function and its parameters and number of neurons in the reservoir can be determined by trial and error or expert view.

2.5. Validation process

In order to evaluate and validate the proposed approach accuracy and root mean square error (RMSE) success rates, which are given below, were employed for classification and regression purposes, respectively. Furthermore, the processing duration was used to show the computational cost of the proposed approach. An Intel Core i7-2600 CPU, 3.4 GHz, 4 GB RAM, PC was used for calculations.

$$Accuracy\ \ (\%) = 100 * \frac{\#\ True\ classified\ samples}{\#\ All\ samples}\ \% \tag{17}$$

$$RMSE = \sqrt{E\left[(f-y)^2\right]} \tag{18}$$

where E is the expected value. f and y are desired and obtained values, respectively. Proposed approach were compared with some other randomized ANNs, which are the random weight neural network (RNN) [52], the random vector functional links

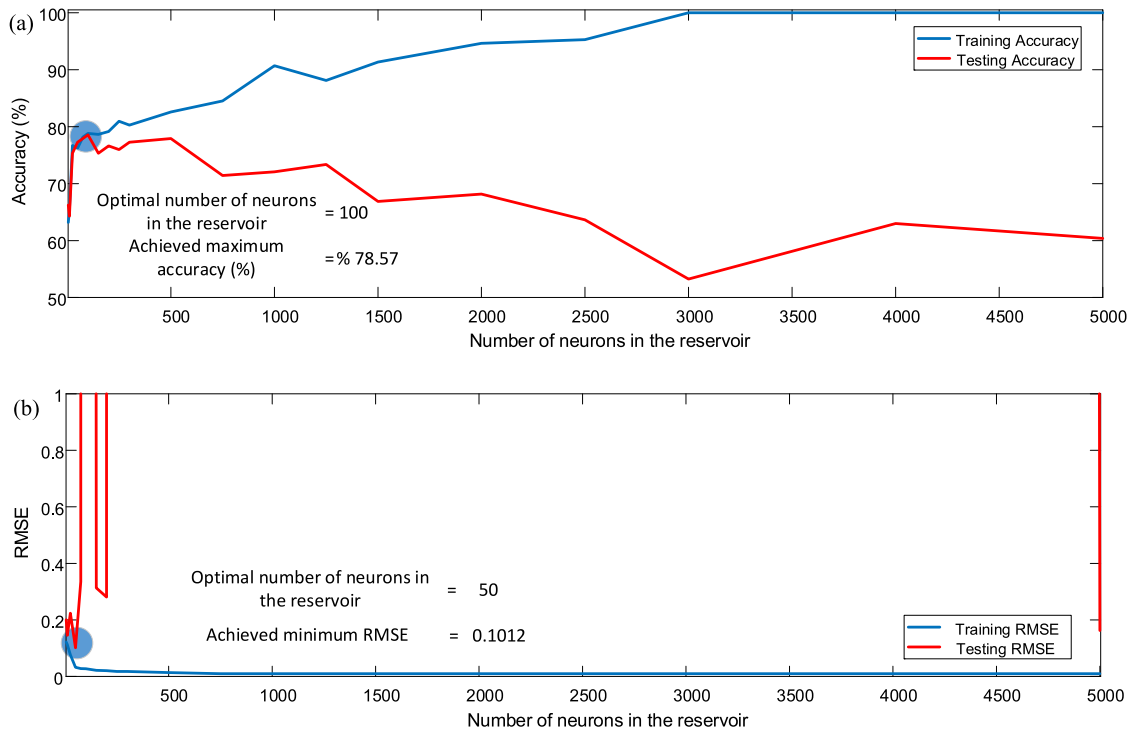

Fig. 4. Achieved success rates in (a) Pima Indian diabetes, and (b) Machine CPU.

(RVFL) [53], the extreme learning machine (ELM) [29], stochastic ELM (S-ELM) [31], and pruned stochastic ELM (PS-ELM) [31]. Details of the employed validation process are given below.

1. The positions of the samples in the dataset were not changed,
2. Samples were normalized between −1 to 1.
3. 5-folds cross-validation procedure was employed in order to decrease the computational complexity since 60 different benchmark datasets were employed in the simulations.
4. Each of the other employed methods was used in the same data partitions.
5. In RNN, RVFL, and ELM, the network structure was optimized based on trials (testes activation functions: sigmoid, sine, triangular basis, and radial basis; tested number of neurons in the hidden layer: 2, 3, 4, 5, 10, 20, 30, 40, 50, 60, 70, 80, 90, and 100).

Please note that S-ELM, PS-ELM, and the proposed approach do not require any optimization stage. In S-ELM and PS-ELM, there is a group of parallel ANNs, while in the proposed approach (RC-ELM), there is only one network.

6. Employed activation functions in S-ELM, PS-ELM, and RC-ELM are linear, positive linear, symmetric saturating linear, hard-limit, symmetric hard-limit, sigmoid, log-sigmoid, hyperbolic tangent sigmoid, hyperbolic tangent, sine, cosine, radial basis, triangular basis, Gaussian and multi-quadratic.

3. Results and discussion

3.1. Analyzing the effect of the number of neurons in the reservoir

In order to assess the effect of the number of neurons in the reservoir over the success of the proposed approach, the following experimental procedure was employed in two datasets, which are Pima Indian Diabetes (classification) and machine CPU (regression).

1. The number of neurons in the hidden layer were allocated as 5, 10, 25, 50, 75, 100, 150, 200, 250, 300, 500, 750, 1000, 1250, 1500, 2000, 2500, 3000, 4000, and 5000.
2. The ratio of the links between input neurons with the neurons in the reservoir was assigned randomly,
3. The activation functions and the parameters that belong to the activation functions were assigned arbitrary,
4. The success rates reported according to 5-folds cross-validation,

Here, the reason behind assigning arbitrary activation functions and the ratio of the links between input neurons with the neurons in the reservoir is to decrease the computational cost of optimizing these parameters. Achieved accuracies (%) in Pima Indian diabetes dataset and RMSE in machine CPU datasets were given in Fig. 4(a) and (b), respectively.

The increase in the number of neurons in the reservoir yields rise in the training success, but on the other hand, it yields a decrease in the testing accuracy [31]. These achieved results are well suited to the literature findings. The increase in the neurons in the hidden layer causes extreme learning of an ANN. The decrease in the generalization capacity of an ANN that extremely learns yields a decrease in the obtained test success (i.e., decrease in the obtained classification accuracy or increase in the obtained regression RMSE). Furthermore, in order to investigate the correlation between the number of neurons in the reservoir and the computational cost, the process duration in both Pima Indian diabetes and machine CPU datasets are summarized in Fig. 5.

As seen in Fig. 5, the increase in the number of neurons in the reservoir yields an expansion in the duration of both of the training and testing stages. I.e., the computational cost of the artificial neural networks that has a higher number of neurons in the reservoir is higher than the one that has a lower number of neurons in the reservoir.

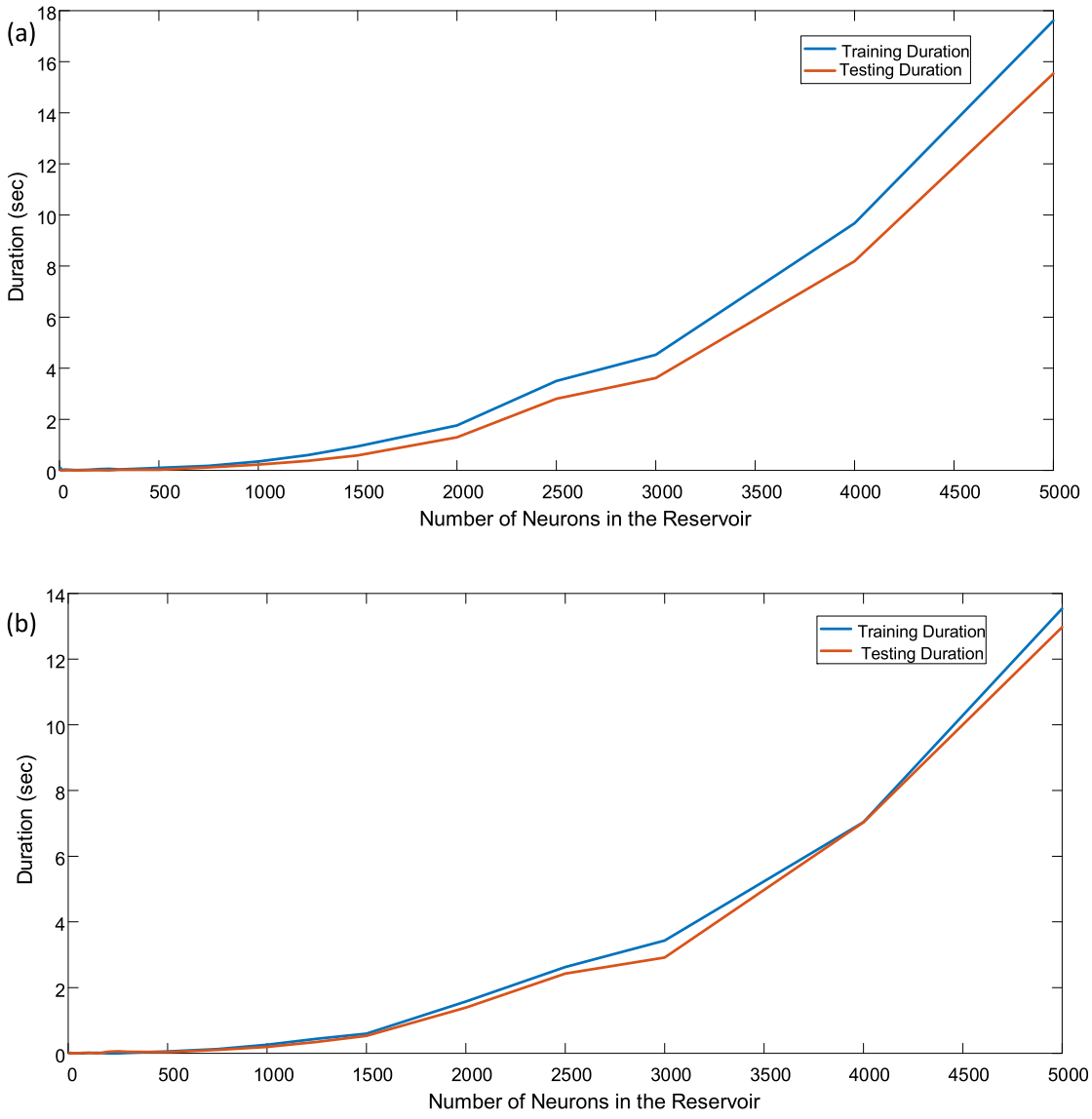

Fig. 5. Durations (sec) in Pima Indian diabetes and machine CPU datasets based on the number of neurons in the reservoir: **(a)** Pima Indian diabetes, and **(b)** machine CPU.

3.2. Analyzing the effect of the ratio of the linked input neurons with reservoir neurons

In order to assess the effect of the ratio of the linked input neurons with reservoir neurons, the following experimental procedure was employed in two datasets, which are Pima Indian Diabetes (classification) and machine CPU (regression).

1. The ratio of linked input neurons with the reservoir neurons was assigned from 0.01 to 0.99 by increasing 0.01 in each step,
2. The number of neurons in the reservoir were assigned constant, that is 100 neurons in Pima Indian diabetes and 50 neurons in the machine CPU,
3. The activation functions and the parameters that belong to the activation functions were assigned arbitrary,
4. The success rates reported according to 5-folds cross-validation

Here, the reason behind assigning arbitrary activation functions is to decrease the computational cost of optimizing these parameters. Achieved success rates in Pima Indian diabetes and machine CPU are given in Fig. 6(a), and (b), respectively.

As seen in Fig. 6, obtained success rates in both datasets are highly related to the ratio of the linked input neurons with reservoir neurons, the number of neurons in the reservoir. The effect of the ratio of the linked input neurons with reservoir neurons is the major reason behind the fluctuations in Fig. 4. Determining the optimal ratio of the linked input neurons with reservoir neurons is a drawback of the proposed approach. This parameter can be assigned by trial and error or by an expert view. In order to assess the effect of the ratios of the linked input neurons with reservoir neurons on the computational cost, the process durations are given in Fig. 7.

Since the number of neurons in the reservoir is less, the relationship between the ratio of linked input neurons with the reservoir neurons and the training and test durations could not be investigated as seen in Fig. 7. Please note that it was expected that the increase in these ratios yields a more complex network (i.e., the number of links is increased), therefore, the networks that have higher ratios must take a longer duration to process.

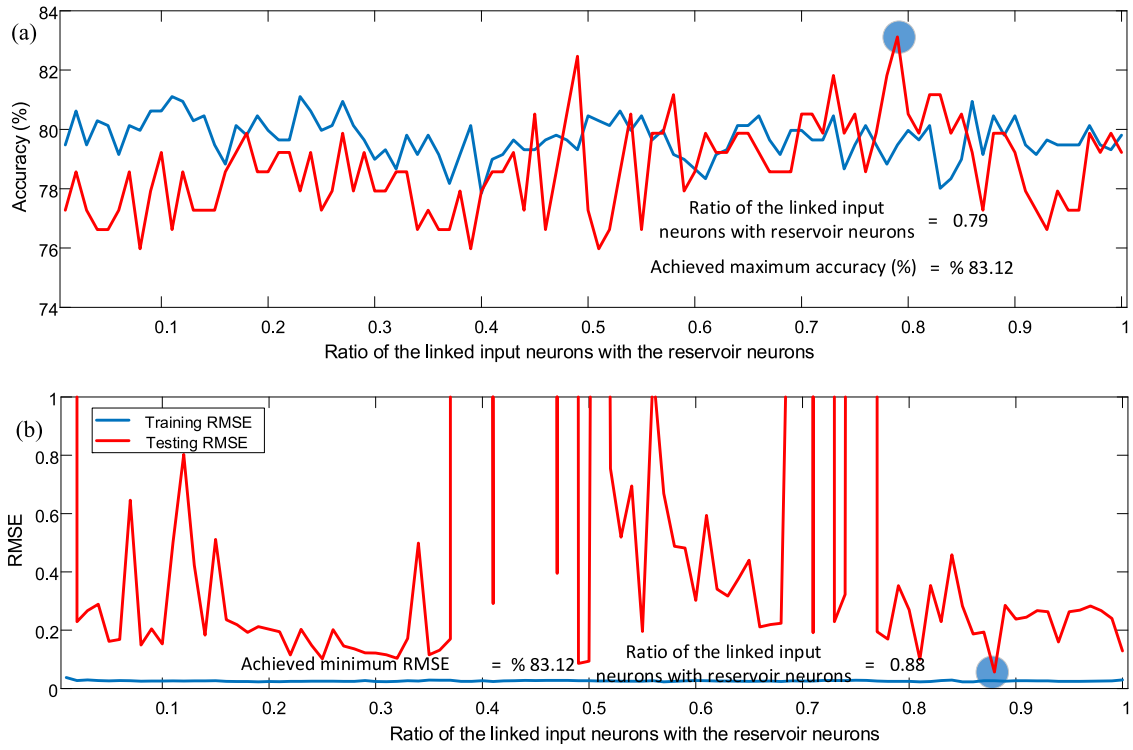

Fig. 6. Achieved success rates based on the ratio of the linked input neurons with reservoir neurons in **(a)** Pima Indian diabetes, and **(b)** machine CPU.

Table 2
Obtained summarized results in 500 trials of the same method.

	Parameter	Minimum	Mean	Maximum	Standard deviation
Diabetes	Training Accuracy (%)	74.560	80.558	83.909	1.547
	Testing Accuracy (%)	72.338	75.756	78.312	1.038
	Training Duration (sec)	0.000	0.089	0.200	0.047
	Testing Duration (sec)	0.000	0.026	0.075	0.017
	# neurons in reservoir	50.0	220.552	300.0	82.869
	Input link ratio	0.200	0.350	0.500	0.088
Machine CPU	Training RMSE	0.017	0.026	0.052	0.006
	Testing RMSE	0.052	0.071	0.089	0.018
	Training Duration (sec)	0.000	0.016	0.081	0.017
	Testing Duration (sec)	0.000	0.009	0.063	0.011
	# neurons in reservoir	50.0	98.820	154.0	31.317
	Input link ratio	0.201	0.356	0.500	0.088

3.3. Analyzing the effect of randomization in RC-ELM

In order to analyze the effect of the randomization in RC-ELM over the obtained success rates, the following experimental procedure was employed in two datasets, which are Pima Indian Diabetes (classification) and machine CPU (regression).

1. The ratio of linked input neurons with the reservoir neurons was assigned randomly,
2. The number of neurons in the reservoir were assigned randomly,
3. The activation functions and the parameters that belong to the activation functions were assigned randomly,
4. The success rates reported according to 5-folds cross-validation,

In order to assess the effect of employing randomized parameters, the RC-ELM that had randomized parameters were employed 500 times. Obtained success rates in Pima Indian Diabetes (classification) and machine CPU (regression) are summarized in Table 2.

Although the parameters in RC-ELM were assigned arbitrary, the difference between the obtained maximum and minimum success rates are less enough (see Table 2). Therefore, it can be said that the cost of randomization is acceptable with respect to searching the optimal network structure in a limited group of alternatives.

Reported testing accuracies in Pima Indian diabetes by support vector machines (SVM), backpropagation (BP), ELM, H-ELM, Sa-ELM, and CFWNN-ELM were 77.31 [29], 74.73 [29], 77.57 [29], 80.47 [61], 79.55 [62], and 78.02 [62], respectively. As seen in Table 2, achieved minimum, mean, and maximum accuracies in RC-ELM were 72.338, 75.756, and 78.312, respectively. Furthermore, reported testing RMSEs in Machine CPU by SVM, BP, ELM, B-ELM, I-ELM, EM-ELM, and EI-ELM were 0.0811 [29], 0.0826 [29], 0.0539 [29], 0.520 [63], 0.1261 [63], 0.0719 [63], and 0.0653 [63], respectively, while achieved minimum, mean and maximum RMSEs by RC-ELM were 0.052, 0.071, and 0.089, respectively. Based on these literature findings, it can be said that acceptable success rates were obtained by RC-ELM.

3.4. Obtained success rates in RC-ELM

Proposed approach (employed according to the procedure given in Section 3.3) and some other randomized ANNs (employed according to validation strategy given in Section 2.5)

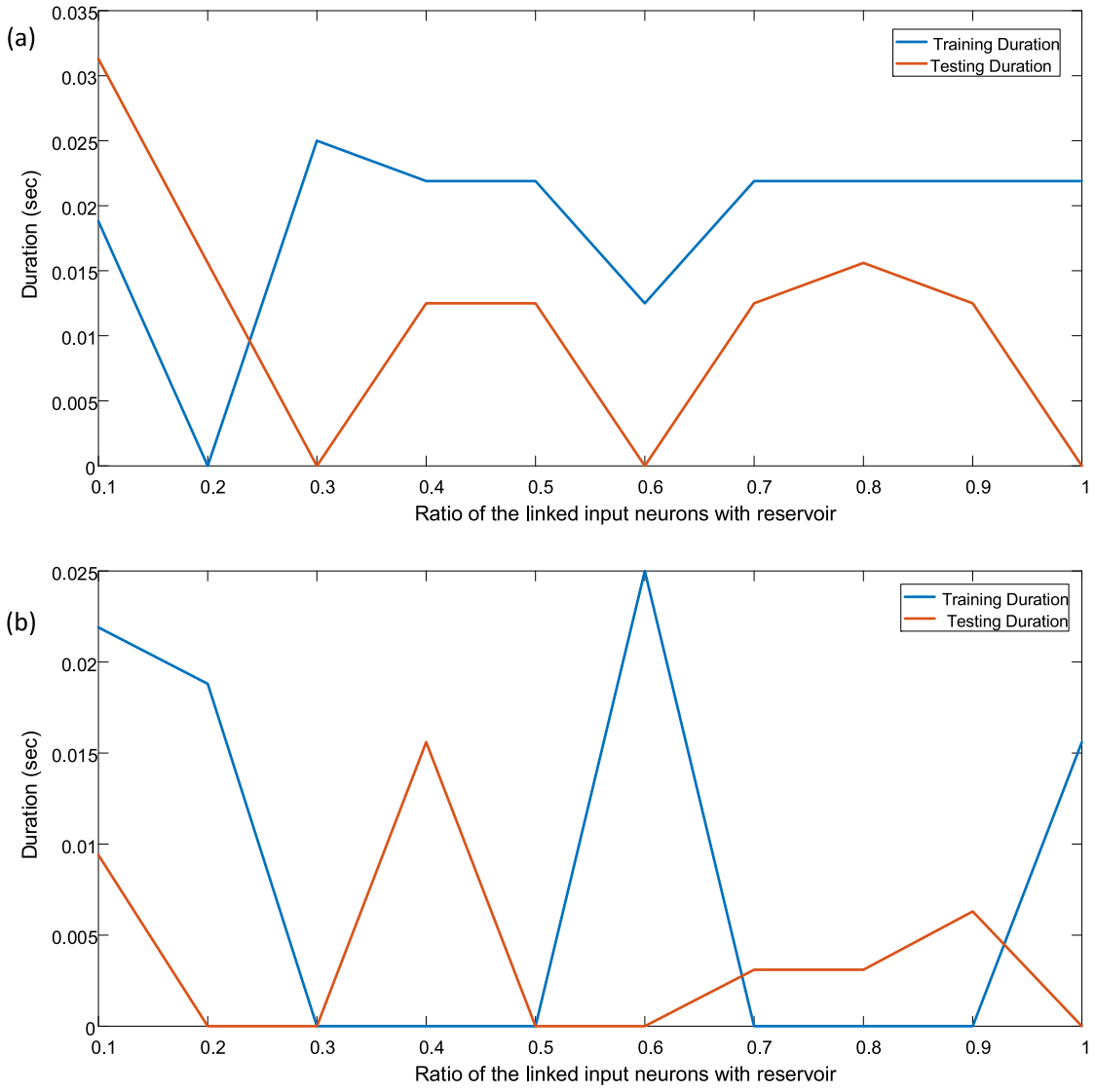

Fig. 7. Process durations (sec) according to the ratio of the linked input neurons with reservoir: **(a)** Pima Indian diabetes, and **(b)** machine CPU.

were employed to the benchmark datasets and obtained success rates in regression and classification benchmark datasets are summarized in Tables 3 and 4, respectively.

As seen in both of Tables 3 and 4, achieved accuracies by RC-ELM is generally higher than or quite near with obtained success rates by other employed datasets. The reason behind this success may be higher modeling ability of RC-ELM. Furthermore, time durations in regression and classification datasets are given in Tables 5 and 6, respectively.

4. Discussion

Given results in Tables 3–6, are summarized in Table 7 based on employed method. As seen in Table 7, although the best mean training success rates were obtained by PS-ELM, the best mean testing success rates were obtained by the proposed approach. The reason behind this success may be its higher ability in providing nonlinearity [31], because of the randomly assigning activation function that has arbitrarily determined parameters in each particular link between neurons in the reservoir and the neurons in the output layer.

The main contributions of this paper to the literature are (1) a novel reservoir computing approach, which can be employed for a non-sequential dataset successfully, has been proposed (2) the hidden layer in ELM has been used as a reservoir of neurons, and (3) acceptable success rates were achieved without having an optimization stage. Furthermore, the computational cost of the RC-ELM is slightly higher or similar to the computational cost of ELM, RNN, and RVFL. Since RC-ELM does not require any optimization stage (i.e., many time-consuming trials), it can be easily said that the total computational cost of RC-ELM is much lower than ELM, RNN, and RVFL. On the other hand, RC-ELM has much training speed than S-ELM and PS-ELM. Because in S-ELM and PS-ELM a group of networks is trained at the same time and the best one chosen as an optimal network based on a winner takes all strategy [31].

Please note that the trial based optimization approach, which is the most employed optimization strategy, has a major disadvantage, it searches the optimal number of neurons in the hidden layer/reservoir and the activation functions inside a predefined group. Therefore, it does not guarantee to determine the optimal network structure. On the other hand, in RC-ELM, the parameters can be optimized by trials or they can be assigned randomly. But optimizing RC-ELM by trial is a time-consuming process because different activation functions can be employed in each of the links between input and reservoir layer. In other words, in RC-ELM

Table 3
Obtained RMSE by ELM, RVFL, RNN, S-ELM, PS-ELM, and RC-ELM.

Datasets	Training RMSE						Testing RMSE					
	ELM	RFVL	RNN	S-ELM	PS-ELM	RC-ELM	ELM	RFVL	RNN	S-ELM	PS-ELM	RC-ELM
Approximate Sinc	**0.57**	**0.57**	**0.57**	**0.57**	**0.57**	**0.57**	**0.57**	**0.57**	**0.57**	**0.57**	0.58	0.58
Forest Fire	0.06	**0.05**	0.06	0.06	**0.05**	**0.05**	0.05	0.06	0.05	**0.04**	0.05	**0.04**
CASP 5-9	**0.04**	**0.04**	**0.04**	**0.04**	**0.04**	**0.04**	**0.04**	**0.04**	**0.04**	**0.04**	**0.04**	**0.04**
Istanbul Stock Exchange	0.14	0.14	0.13	0.14	**0.11**	**0.11**	0.15	**0.14**	**0.14**	**0.14**	0.83	**0.14**
Ailerons	**0.07**	0.08	**0.07**	**0.07**	**0.07**	**0.07**	0.08	0.08	0.08	0.08	0.08	**0.04**
Delta Ailerons	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
Auto-Price	0.06	0.1	0.06	0.07	**0.02**	0.04	0.12	**0.09**	0.11	0.1	0.57	0.11
Bank-8FM	**0.04**	0.05	**0.04**	0.05	0.07	**0.04**	**0.05**	**0.05**	**0.05**	**0.05**	0.07	0.09
Boston Housing	0.09	0.11	0.08	0.07	0.06	**0.05**	0.14	**0.1**	**0.1**	0.13	0.19	0.12
Breast Cancer	0.25	0.27	0.26	0.25	**0.13**	0.16	0.29	0.24	0.29	0.28	1.76	**0.23**
California Housing	**0.12**	0.15	**0.12**	0.13	**0.12**	**0.12**	0.13	0.14	0.13	0.13	0.21	**0.12**
Census-8L	**0.06**	0.07	**0.06**	0.07	0.07	0.07	**0.07**	**0.07**	**0.07**	**0.07**	0.09	0.13
Census-8H	**0.08**	0.09	**0.08**	**0.08**	**0.08**	**0.08**	**0.08**	0.09	**0.08**	**0.08**	**0.08**	0.09
Census-16L	**0.07**	0.08	**0.07**	0.08	**0.07**	**0.07**	**0.07**	0.08	**0.07**	0.08	0.1	0.08
Census-16H	**0.08**	0.09	**0.08**	**0.08**	**0.08**	**0.08**	0.08	0.09	0.08	0.08	0.08	**0.05**
CPU-Small	**0.03**	0.05	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	0.05	0.04	0.04	0.04	0.06
CPU	**0.04**	0.06	**0.04**	0.05	0.05	0.06	**0.04**	0.06	**0.04**	0.05	0.07	0.05
Diabetes Child	0.18	0.1	0.09	0.07	0.07	**0.06**	0.2	**0.09**	0.1	0.19	1.46	0.13
Delta Elevators	**0.11**	**0.11**	**0.11**	**0.11**	**0.11**	**0.11**	**0.11**	**0.11**	**0.11**	**0.11**	**0.11**	**0.11**
Elevators	0.07	0.08	0.07	0.08	0.05	**0.04**	0.08	0.08	**0.07**	0.08	**0.07**	0.08
Kinematics	0.11	0.11	**0.1**	0.11	0.11	0.12	0.11	0.11	**0.1**	0.11	0.11	0.11
Machine-CPU	0.03	0.06	0.04	0.04	**0.02**	**0.02**	0.13	0.05	0.08	0.07	0.64	**0.02**
Puma-8NH	0.35	0.35	0.35	**0.34**	**0.34**	0.35	0.36	**0.34**	0.36	0.35	**0.34**	0.36
Puma-32H	**0.3**	**0.3**	**0.3**	**0.3**	**0.3**	**0.3**	0.3	0.3	0.3	0.3	0.31	**0.21**
Pyrimidines	0.06	0.09	0.09	0.05	**0.03**	0.05	0.11	0.08	0.12	0.12	0.22	**0.06**
Servo	0.08	0.17	**0.05**	0.06	0.08	0.08	0.1	0.16	0.1	0.1	0.11	**0.07**
Stocks	0.02	0.07	0.02	0.02	**0.01**	**0.01**	0.07	**0.04**	0.06	0.05	0.53	0.07
Triazines	0.15	0.16	0.14	0.13	0.03	**0.08**	0.17	0.14	0.17	0.17	1.71	**0.13**
Energy Efficiency: cooling	0.05	0.07	0.06	0.07	0.05	**0.04**	0.1	0.07	0.07	0.07	0.09	**0.06**
Energy Efficiency: heating	**0**	0.01	**0**	**0**	**0**	**0**	**0**	0.01	**0**	**0**	**0**	0.01

Table 4
Obtained accuracy (%) by ELM, RVFL, RNN, S-ELM, PS-ELM, and RC-ELM.

Datasets	Training Accuracy (%)						Testing Accuracy (%)					
	ELM	RFVL	RNN	S-ELM	PS-ELM	RC-ELM	ELM	RFVL	RNN	S-ELM	PS-ELM	RC-ELM
Lithuanian	97.25	96.68	96.80	96.40	97.00	**97.43**	**95.70**	95.50	95.40	94.70	93.80	95.40
Highleyman	93.78	87.13	92.88	**94.93**	93.98	93.30	90.40	76.50	88.30	**91.60**	89.20	91.40
Banana Shaped	98.15	92.53	98.15	98.13	97.43	**98.53**	97.50	89.10	**97.80**	97.40	95.80	97.30
Spherical	84.90	84.23	84.93	86.25	**87.15**	85.18	79.90	76.50	79.90	**80.50**	78.10	79.90
Multi-Class	92.30	78.70	92.08	**92.35**	91.65	91.20	**41.90**	21.70	41.40	41.10	38.60	41.40
Liver	70.69	71.80	71.46	73.52	**79.96**	78.15	70.60	71.45	70.77	**72.65**	63.42	71.15
Pima Indian Diabetes	78.14	77.65	78.93	78.73	**84.23**	81.63	76.23	76.75	77.66	**77.79**	73.90	76.45
Hepatitis	81.56	77.19	**100.00**	82.50	98.13	90.63	51.25	58.75	60.00	**66.25**	60.00	65.00
Banana	90.57	89.17	**90.86**	90.72	90.39	90.50	84.17	78.06	79.55	**84.32**	70.91	83.13
Image Segmentation	95.58	88.57	**95.87**	95.08	94.55	95.28	**89.50**	78.10	87.73	88.21	86.57	88.51
Satellite Image	87.15	80.62	86.41	86.43	**87.92**	86.01	85.08	79.49	85.03	84.94	**86.25**	85.37
Statlog (Shuttle)	99.44	95.81	98.94	**99.46**	99.17	94.35	99.35	95.77	98.86	**99.40**	99.16	98.11
Abalone	29.87	26.71	30.14	30.62	30.83	**31.56**	26.30	25.15	26.42	**26.54**	26.37	26.49
Wine	99.72	96.34	**100.00**	99.72	99.86	**100.00**	93.33	87.78	90.56	94.44	92.22	**97.44**
Breast tissue	62.59	66.59	73.41	82.59	**93.65**	86.35	1.90	0.95	4.76	21.90	16.19	**22.43**
Cardiotocography	95.70	92.30	95.13	94.51	94.97	**95.90**	**92.89**	89.74	92.19	91.81	92.14	92.19
Skin segmentation	99.33	98.56	**99.52**	99.35	99.32	98.87	86.97	78.15	88.17	**90.66**	88.48	89.50
Seeds	97.62	92.74	97.26	97.62	**99.40**	99.17	91.90	81.90	**93.33**	90.48	84.76	92.10
EEG eye state	60.75	57.20	59.83	68.31	**80.79**	80.18	41.84	39.07	**44.17**	35.31	31.09	39.78
Seismic Bumps	93.41	93.42	93.42	93.42	**94.09**	93.67	93.38	93.42	93.42	93.42	91.76	**94.42**
Banknote authentication	99.96	98.63	**100.00**	99.98	**100.00**	99.98	99.56	97.30	99.85	**100.00**	**100.00**	99.49
Balance Scale	92.92	90.68	91.36	93.00	**94.72**	93.64	**89.76**	88.96	87.20	88.80	79.36	89.72
Acute Inflammations	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	99.50
Dermatology	99.11	98.29	98.36	98.70	**99.59**	98.36	96.99	96.99	96.16	**97.53**	93.97	96.88
Diabetic Retinopathy Debrecen	76.44	66.86	75.90	75.44	**79.39**	78.33	70.87	64.70	72.96	72.00	70.43	**73.35**
Fertility	88.00	88.00	88.00	89.00	**97.50**	95.25	**88.00**	**88.00**	**88.00**	85.00	79.00	87.20
Glass	**94.50**	59.65	93.92	89.12	83.98	84.09	22.33	9.77	24.65	**31.63**	27.44	29.77
Haberman	75.76	73.88	74.69	76.98	**83.92**	81.06	74.43	73.77	72.79	73.44	57.70	**74.69**
Hayes-Roth	91.32	64.15	90.75	**93.02**	90.75	85.47	**75.38**	46.15	69.23	74.62	63.85	74.31
QSAR biodegradation	88.70	84.72	88.36	88.65	**90.26**	88.36	79.72	72.13	77.91	**79.91**	77.91	77.82

Table 5
Process durations in ELM, RVFL, RNN, S-ELM, PS-ELM, and RC-ELM in regression datasets.

Datasets	Training time (sec)						Testing time (sec)					
	ELM	RFVL	RNN	S-ELM	PS-ELM	RC-ELM	ELM	RFVL	RNN	S-ELM	PS-ELM	RC-ELM
Approximate Sinc	**0.00**	0.01	0.02	3.38	1.18	0.03	**0.00**	0.03	**0.00**	0.56	0.04	0.07
Forest Fire	**0.00**	**0.00**	**0.00**	0.43	0.48	0.05	**0.00**	0.01	**0.00**	0.22	0.01	0.03
CASP 5-9	1.72	**0.69**	1.76	34.76	45.78	1.74	**0.07**	0.10	0.09	4.92	1.03	0.19
Istanbul Stock Exchange	**0.00**	0.01	0.02	0.58	0.84	0.02	**0.00**	**0.00**	**0.00**	0.28	0.03	0.01
Ailerons	0.10	0.08	**0.06**	4.32	6.41	0.07	**0.00**	**0.00**	**0.00**	0.73	0.11	0.04
Delta Ailerons	**0.24**	0.27	0.29	9.36	13.81	0.28	**0.02**	0.06	0.04	1.69	0.26	0.10
Auto-Price	**0.00**	0.01	**0.00**	0.31	0.27	**0.00**	**0.00**	**0.00**	**0.00**	0.21	0.01	**0.00**
Bank-8FM	0.10	**0.09**	0.13	3.69	4.63	0.15	**0.00**	**0.00**	**0.00**	0.69	0.09	0.08
Boston Housing	**0.00**	0.03	**0.00**	0.60	0.57	0.05	**0.00**	**0.00**	**0.00**	0.26	0.01	0.01
Breast Cancer	0.01	**0.00**	**0.00**	0.49	0.48	**0.00**	**0.00**	**0.00**	**0.00**	0.20	0.01	0.01
California Housing	0.26	0.33	**0.18**	16.71	15.91	0.43	0.03	0.06	**0.01**	2.55	0.30	0.13
Census-8L	0.82	**0.34**	0.71	22.56	16.73	0.68	0.09	0.05	**0.04**	3.06	0.32	0.10
Census-8H	0.72	**0.38**	0.79	15.44	23.22	0.64	0.03	0.05	**0.02**	2.48	0.36	0.07
Census-16L	0.82	**0.35**	0.82	18.60	23.16	0.67	0.06	**0.05**	0.06	2.56	0.37	0.13
Census-16H	0.79	**0.38**	0.81	16.26	22.36	0.69	0.06	0.06	**0.05**	2.60	0.45	**0.05**
CPU-Small	0.21	0.11	**0.06**	3.12	4.63	0.23	**0.00**	0.06	0.01	0.53	0.08	0.10
CPU	0.17	**0.14**	0.15	2.21	3.94	0.18	**0.00**	**0.00**	0.03	0.46	0.05	0.10
Diabetes Child	**0.00**	**0.00**	**0.00**	0.05	0.01	0.01	**0.00**	**0.00**	**0.00**	0.06	**0.00**	**0.00**
Delta Elevators	0.19	**0.06**	0.10	3.77	3.05	0.27	**0.00**	**0.00**	0.01	0.65	0.06	0.07
Elevators	**0.20**	0.27	0.36	6.58	13.79	0.40	0.06	0.04	0.06	0.99	0.16	0.16
Kinematics	0.19	**0.12**	0.14	1.96	2.59	0.24	0.01	0.01	**0.00**	0.38	0.06	0.08
Machine-CPU	**0.00**	**0.00**	0.01	0.19	0.13	0.01	**0.00**	**0.00**	**0.00**	0.09	0.01	**0.00**
Puma-8NH	0.14	**0.10**	0.13	2.89	2.99	0.24	**0.00**	0.02	**0.00**	0.52	0.05	0.07
Puma-32H	**0.06**	0.09	0.09	1.74	2.53	0.10	**0.02**	**0.02**	**0.02**	0.29	0.05	0.05
Pyrimidines	**0.00**	**0.00**	**0.00**	0.07	0.05	**0.00**	**0.00**	**0.00**	**0.00**	0.06	**0.00**	**0.00**
Servo	**0.00**	**0.00**	**0.00**	0.20	0.09	0.01	**0.00**	**0.00**	**0.00**	0.10	**0.00**	**0.00**
Stocks	0.02	**0.00**	0.01	0.32	0.54	0.06	**0.00**	**0.00**	**0.00**	0.10	0.03	0.01
Triazines	0.01	**0.00**	**0.00**	0.23	0.42	0.03	**0.00**	**0.00**	**0.00**	0.11	0.01	0.01
Energy Efficiency: cooling	0.01	**0.00**	**0.00**	0.30	0.30	0.06	0.01	**0.00**	**0.00**	0.19	0.01	0.01
Energy Efficiency: heating	**0.00**	**0.00**	0.01	0.29	0.33	0.05	**0.00**	**0.00**	**0.00**	0.15	**0.00**	0.03

Table 6
Process durations in ELM, RVFL, RNN, S-ELM, PS-ELM, and RC-ELM in classification datasets.

Datasets	Training time						Testing time					
	ELM	RFVL	RNN	S-ELM	PS-ELM	RC-ELM	ELM	RFVL	RNN	S-ELM	PS-ELM	RC-ELM
Lithuanian	0.03	0.03	**0.00**	0.44	0.50	0.07	**0.00**	**0.00**	**0.00**	0.14	**0.00**	**0.00**
Highleyman	0.03	**0.01**	**0.01**	0.38	0.25	0.08	**0.00**	**0.00**	0.01	0.11	**0.00**	0.03
Banana Shaped	0.01	0.02	**0.00**	0.29	0.24	0.07	**0.00**	**0.00**	**0.00**	0.19	0.01	0.03
Spherical	**0.01**	0.03	0.02	0.36	0.63	0.08	**0.00**	**0.00**	**0.00**	0.20	0.03	0.03
Multi-Class	0.03	**0.00**	0.01	0.46	0.33	0.05	**0.00**	0.01	0.01	0.13	**0.00**	0.02
Liver	0.01	**0.00**	**0.00**	0.41	0.28	0.06	**0.00**	**0.00**	**0.00**	0.20	**0.00**	0.03
Pima Indian Diabetes	0.01	**0.00**	**0.00**	0.22	0.56	0.05	**0.00**	0.01	**0.00**	0.19	**0.00**	0.04
Hepatitis	**0.00**	**0.00**	0.02	0.11	0.08	0.00	0.01	**0.00**	**0.00**	0.08	0.03	**0.00**
Banana	**0.08**	0.18	0.11	2.31	1.99	0.27	**0.00**	0.01	**0.00**	0.43	0.06	0.07
Image Segmentation	0.05	**0.03**	**0.03**	0.83	1.11	0.15	**0.01**	0.03	0.03	0.28	**0.01**	0.08
Satellite Image	**0.13**	0.16	0.15	2.34	4.28	0.34	**0.00**	0.01	**0.00**	0.42	0.09	0.10
Statlog (Shuttle)	2.10	1.21	2.28	3.11	1.92	**0.07**	0.09	0.19	0.14	0.37	0.33	**0.00**
Abalone	**0.01**	0.08	0.06	1.48	0.96	0.23	0.03	0.01	**0.00**	0.41	**0.00**	0.06
Wine	**0.00**	**0.00**	0.02	0.19	0.11	**0.00**	**0.00**	**0.00**	**0.00**	0.13	**0.00**	0.03
Breast tissue	**0.00**	**0.00**	**0.00**	0.17	0.12	0.02	**0.00**	**0.00**	0.01	0.09	**0.00**	0.02
Cardiotocography	0.06	0.05	**0.03**	0.59	0.53	0.11	**0.00**	**0.00**	0.01	0.20	0.01	0.06
Skin segmentation	2.30	4.19	6.66	0.83	**0.76**	9.57	0.20	0.76	0.44	0.15	**0.13**	1.12
Seeds	**0.00**	0.01	0.00	0.24	0.06	0.01	**0.00**	**0.00**	**0.00**	0.06	**0.00**	**0.00**
EEG eye state	0.02	0.12	0.00	3.83	7.63	0.45	0.01	0.03	**0.00**	0.73	0.10	0.05
Seismic Bumps	0.02	0.01	0.00	1.14	1.74	0.10	**0.00**	0.01	**0.00**	0.19	0.04	0.06
Banknote authentication	0.01	0.03	0.03	0.35	0.58	0.07	**0.00**	**0.00**	**0.00**	0.19	0.03	0.04
Balance Scale	0.00	0.01	0.00	0.35	0.40	0.05	**0.00**	**0.00**	**0.00**	0.10	0.04	0.03
Acute Inflammations	0.00	0.00	0.00	0.18	0.05	0.00	**0.00**	**0.00**	**0.00**	0.08	**0.00**	**0.00**
Dermatology	0.01	0.01	0.00	0.38	0.36	0.03	**0.00**	**0.00**	**0.00**	0.10	0.01	**0.00**
Diabetic Retinopathy Debrecen	0.00	0.03	0.01	0.48	0.78	0.09	**0.00**	**0.00**	**0.00**	0.11	0.03	0.03
Fertility	0.00	0.00	0.00	0.18	0.08	0.01	0.01	**0.00**	**0.00**	0.09	**0.00**	**0.00**
Glass	0.02	0.00	0.01	0.19	0.13	0.03	0.01	**0.00**	**0.00**	0.09	0.01	0.00
Haberman	0.01	0.00	0.00	0.19	0.18	0.03	**0.00**	**0.00**	**0.00**	0.10	**0.00**	0.03
Hayes-Roth	0.00	0.01	0.00	0.24	0.05	0.01	**0.00**	**0.00**	**0.00**	0.05	**0.00**	**0.00**
QSAR biodegradation	0.00	0.00	0.03	0.40	0.73	0.05	**0.00**	**0.00**	**0.00**	0.20	**0.00**	0.03

Table 7
Summarized results.

	Method	Success rate		Time (sec)	
		Training	Testing	Training	Testing
Regression	ELM	0.110	0.127	0.226	0.015
	RFVL	0.122	0.117	**0.129**	0.021
	RNN	0.107	0.119	0.222	**0.015**
	S-ELM	0.107	0.123	5.714	0.923
	PS-ELM	**0.094**	0.351	7.041	0.132
	RC-ELM	0.097	**0.113**	0.246	0.057
Classification	ELM	87.17	76.24	**0.165**	**0.012**
	RFVL	82.29	71.05	0.207	0.036
	RNN	87.91	76.14	0.316	0.022
	S-ELM	88.15	77.55	0.756	0.194
	PS-ELM	**90.49**	73.61	0.914	0.032
	RC-ELM	89.08	**77.67**	0.404	0.066

the number of free parameters, which can be optimized, is much higher than ELM and other randomized ANNs.

5. Conclusion

In this paper, a novel approach, which was built on reservoir computing and extreme learning machine, was proposed. In this approach, each of the parameters and the structure of the network were assigned randomly. Therefore, higher nonlinearity can be provided in the proposed approach. Obtained results by the proposed approach were compared with some other popular randomized ANNs and literature findings. The proposed approach found successful enough in terms of both achieved success rates and process speeds. Furthermore, the traditional reservoir computing methods, which are based on delays, showed high success in the time-based datasets and they are not employed in non-sequential datasets. On the other hand, the novel reservoir computing approach showed high successes in non-sequential (batch) datasets.

CRediT authorship contribution statement

Ömer Faruk Ertuğrul: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing - original draft, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Ö.F. Ertuğrul, M.E. Tağluk, A novel machine learning method based on generalized behavioral learning theory, Neural Comput. Appl. 28 (12) (2017) 3921–3939.
[2] I.A. Basheer, M. Hajmeer, Artificial neural networks: Fundamentals, computing, design, and application, J. Microbiol. Methods 43 (1) (2000) 3–31.
[3] Ö.F. Ertuğrul, M.E. Tağluk, A novel machine learning method based on generalized behavioral learning theory, Neural Comput. Appl. (2016).
[4] L.A. Zadeh, Fuzzy logic = computing with words, IEEE Trans. Fuzzy Syst. 4 (2) (1996) 103–111.
[5] M. Abdullahi, M.A. Ngadi, S.I. Dishing, S.M. Abdulhamid, M.J. Usman, A survey of symbiotic organisms search algorithms and applications, in: Neural Computing and Applications, Springer London, 2019.
[6] A. Darwish, A.E. Hassanien, S. Das, A survey of swarm and evolutionary computing approaches for deep learning, Artif. Intell. Rev. (2019).
[7] D. Połap, K. Kęsik, M. Woźniak, R. Damaševičius, Parallel technique for the metaheuristic algorithms using devoted local search and manipulating the solutions space, Appl. Sci. 8 (2) (2018) 293.
[8] R. Damaševičius, M. Woźniak, State flipping based hyper-heuristic for hybridization of nature inspired algorithms, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), in: LNAI, vol. 10245, 2017, pp. 337–346.
[9] G. Deco, E.T. Rolls, R. Romo, Stochastic dynamics as a principle of brain function, Prog. Neurobiol. 88 (1) (2009) 1–16.
[10] P.M.S. Malden, Philosophical Foundations of Neuroscience, 2003.
[11] J. Kleinberg, Jon, The small-world phenomenon, in: Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing - STOC '00, 2000, pp. 163–170.
[12] D.J. Watts, S.H. Strogatz, Collective dynamics of 'small-world' networks, Nature 393 (6684) (1998) 440–442.
[13] J. Travers, S. Milgram, An experimental study of the small world problem, Sociometry 32 (4) (1969) 425.
[14] J.B. Butcher, D. Verstraeten, B. Schrauwen, C.R. Day, P.W. Haycock, Reservoir computing and extreme learning machines for non-linear time-series data analysis, Neural Netw. 38 (2013) 76–89.
[15] H. Jaeger, The 'echo state' approach to analysing and training recurrent neural networks – with an Erratum note 1, GMD Rep. (148) (2010) 1–47.
[16] X. Lin, Z. Yang, Y. Song, Short-term stock price prediction based on echo state networks, Expert Syst. Appl. 36 (3) (PART 2) (2009) 7313–7317.
[17] Y. Xue, L. Yang, S. Haykin, Decoupled echo state networks with lateral inhibition, Neural Netw. 20 (3) (2007) 365–376.
[18] M.H. Tong, A.D. Bickett, E.M. Christiansen, G.W. Cottrell, Learning grammatical structure with echo state networks, Neural Netw. 20 (3) (2007) 424–432.
[19] A. Deihimi, H. Showkati, Application of echo state networks in short-term electric load forecasting, Energy 39 (1) (2012) 327–340.
[20] M. Salmen, P.G. Ploger, Echo state networks used for motor control, in: Proc. 2005 IEEE Int. Conf. Robot. Autom., no. April, 2005, pp. 1953–1958.
[21] M. Cernansky, M. Makula, Feed-forward echo state networks, in: Neural Networks, 2005. IJCNN '05. Proceedings. 2005 IEEE Int. Jt. Conf., vol. 3, 2005, pp. 1479–1482.
[22] D. Prokhorov, Echo state networks: Appeal and challenges, in: Proc. Int. Jt. Conf. Neural Networks, vol. 3, no. Figure 3, 2005, pp. 1463–1466.
[23] N. Chouikhi, B. Ammar, N. Rokbani, A.M. Alimi, PSO-Based analysis of Echo State Network parameters for time series forecasting, Appl. Soft Comput. J. 55 (2017) 211–225.
[24] X. Sun, T. Li, Q. Li, Y. Huang, Y. Li, Deep belief echo-state network and its application to time series prediction, Knowl.-Based Syst. 130 (2017) 17–29.
[25] W. Yao, Z. Zeng, C. Lian, Generating probabilistic predictions using mean–variance estimation and echo state network, Neurocomputing 219 (2016) (2017) 536–547.
[26] C. Gallicchio, A. Micheli, Tree echo state networks, Neurocomputing 101 (2013) 319–337.
[27] A. Rodan, P. Tiňo, Minimum complexity echo state network, IEEE Trans. Neural Netw. 22 (1) (2011) 131–144.
[28] S. xian Lun, X. shuang Yao, H. feng Hu, A new echo state network with variable memory length, Inf. Sci. (Ny). 370–371 (2016) 103–119.
[29] G.-B. Huang, Q. Zhu, C. Siew, Ã, Extreme learning machine: Theory and applications, Neurocomputing 70 (1–3) (2006) 489–501.
[30] Ö. Faruk Ertuğrul, Y. Kaya, A detailed analysis on extreme learning machine and novel approaches based on ELM, Am. J. Comput. Sci. Eng. 1 (5) (2014) 43–50.
[31] Ö.F. Ertuğrul, Two novel versions of randomized feed forward artificial neural networks: Stochastic and pruned stochastic, Neural Process. Lett. (2017).
[32] R.P.W. T. D. M. J. Duin, P. Juszczak, P. Paclik, E. Pekalska, D. de Ridder, PR-Tools 4.0, a Matlab toolbox for pattern recognition, The Netherlands, 2004.
[33] M. Lichman, UCI Machine Learning Repository , Irvine, CA Univ. California, Sch. Inf. Comput. Sci., 2013,.

[34] http://mldata.org/repository/data/viewslug/banana-ida/.
[35] C.S. Guang-bin Huang, Qin-yu Zhu, Extreme learning machine: A new learning scheme of feedforward neural networks, Neurocomputing 70 (2006) 489–501.
[36] http://www.dcc.fc.up.pt/~ltorgo/Regression/DataSets.html.
[37] S. Ortín, et al., A unified framework for reservoir computing and extreme learning machines based on a single time-delayed neuron, Sci. Rep. 5 (July) (2015) 1–11.
[38] N. McDonald, Reservoir computing extreme learning machines using pairs of cellular automata rules, in: 2017 Int. Jt. Conf. Neural Networks, vol. 88, 2017, pp. 2429–2436.
[39] J. Butcher, D. Verstraeten, B. Schrauwen, Extending reservoir computing with random static projections: a hybrid between extreme learning and RC, Artif. Neural (April) (2010) 1–6.
[40] Z. Liu, C.H.U.K. Loo, S. Member, N. Masuyama, Machine for Time Series Prediction, Vol. 4, 2017.
[41] G. Huang, G. Bin Huang, S. Song, K. You, Trends in extreme learning machines: A review, Neural Netw. 61 (2015) 32–48.
[42] G. Dudek, Extreme learning machine as a function approximator: Initialization of input weights and biases, Adv. Intell. Syst. Comput. 403 (2016) 59–69.
[43] Ö.F. Ertuğrul, Forecasting electricity load by a novel recurrent extreme learning machines approach, Int. J. Electr. Power Energy Syst. 78 (2016).
[44] Ö.F. Ertuğrul, Y. Kaya, Smart city planning by estimating energy efficiency of buildings by extreme learning machine, in: 4th International Istanbul Smart Grid Congress and Fair, ICSG 2016, 2016.
[45] Ö.F. Ertuğrul, M. Emin Tağluk, Y. Kaya, R. Tekin, EMG signal classification by extreme learning machine | EMG sinyallerinin agiri?grenme makinesi ile siniflandirilmasi, in: 2013 21st Signal Processing and Communications Applications Conference, SIU 2013, 2013.
[46] Ö.F. Ertuğrul, A novel type of activation function in artificial neural networks: Trained activation function, Neural Netw. 99 (2018).
[47] M. Lukoševičius, H. Jaeger, Reservoir computing approaches to recurrent neural network training, Comput. Sci. Rev. 3 (3) (2009) 127–149.
[48] G. Bin Huang, What are extreme learning machines? filling the gap between Frank Rosenblatt's dream and John von Neumann's puzzle, Cognit. Comput. 7 (3) (2015) 263–278.
[49] G. Bin Huang, An insight into extreme learning machines: Random neurons, random features and kernels, Cognit. Comput. 6 (3) (2014) 376–390.
[50] G. Bin Huang, D.H. Wang, Y. Lan, Extreme learning machines: A survey, Int. J. Mach. Learn. Cybern. 2 (2) (2011) 107–122.
[51] Q.-Y. Zhu, A.K. Qin, P.N. Suganthan, G.-B. Huang, Evolutionary extreme learning machine, Pattern Recognit. 38 (10) (2005) 1759–1763.
[52] W.F. Schmidt, M.A. Kraaijveld, R.P.W. Duin, Feed forward neural networks with random weights, in: IEEE, 11th International Conference on Pattern Recognition, 1992, pp. 1–4.
[53] Y.H. Pao, G.H. Park, D.J. Sobajic, Learning and generalization characteristics of the random vector functional-link net, Neurocomputing 6 (2) (1994) 163–180.
[54] D. Wang, Editorial?: Randomized algorithms for training neural networks, Inf. Sci. (Ny) 364–365 (2016) 126–128.
[55] M. Li, D. Wang, Insights into randomized algorithms for neural networks: Practical issues and common pitfalls, Inf. Sci. (Ny) 382–383 (December) (2017) 170–178.
[56] L. Zhang, P.N. Suganthan, A comprehensive evaluation of random vector functional link networks, Inf. Sci. (Ny) 367–368 (2016) 1094–1105.
[57] K. Hornik, Approximation capabilities of multilayer feedforward networks, Neural Netw. 4 (2) (1991) 251–257.
[58] A. Hernández-Aguirre, C. Koutsougeras, B.P. Buckles, Sample complexity for function learning tasks through linear neural networks, Lecture Notes in Comput. Sci. 2313 (2002) 262–271.
[59] A. Porwal, E.J.M. Carranza, M. Hale, Artificial neural networks for mineral-potential mapping: A Case study from Aravalli Province, Western India, Nat. Resour. Res. 12 (3) (2003) 155–171.
[60] D.P. Das, G. Panda, Active mitigation of nonlinear noise processes using a novel filtered-s LMS algorithm, IEEE Trans. Speech Audio Process. 12 (3) (2004) 313–322.
[61] J. Tang, ChenweiDeng, G.-B. Huang, Extreme learning machine for multilayer perceptron, IEEE Trans. Neural Networks Learn. Syst. (2015) 1–13.
[62] J. Cao, Z. Lin, G. Bin Huang, Self-adaptive evolutionary extreme learning machine, Neural Process. Lett. 36 (3) (2012) 285–305.
[63] Y. Wang, Y. Yuan, X. Yang, Bidirectional extreme learning machine for regression problem and its learning effectiveness, IEEE Trans. Neural Netw. Learn. Syst. 23 (9) (2012) 1498–1505.